ST2 Assignment 1 Part 2 W5 Lab

Task 1: Understanding Recursion algorithm and testing it for simple and complex cases (Fibonacci sequence and factorial computation)

(1)

Test Case 1:

```
The sum of the first 5 natural numbers is 15
The 7th Fibonacci number is 13
The factorial of 5 is 120
```

Test Case 2:

```
The sum of the first 10 natural numbers is 55
The 23rd Fibonacci number is 28657
The factorial of 9 is 362880
```

Test Case 3:

```
The sum of the first 17 natural numbers is 153
The 30th Fibonacci number is 832040
The factorial of 12 is 479001600
```

(2)

```python
'''Basic Iteration Conversion'''
def sum_of_natural_numbers(n):  1 usage
    return n * (n + 1) // 2
```

```python
'''Basic Iteration Conversion 2'''
def fibonacci(n):  1 usage
    a, b = 0, 1
    for i in range(n):
        a, b = b, a + b
    return a
```

```python
'''Basic Iteration Conversion 3'''
def factorial(n):  1 usage
    result = 1
    for i in range(1, n + 1):
        result += i
    return result
```

(3)

As the value of ‘n’ is increased for each recursive function the time it takes to complete slows down, whereas the iterative functions stay fast. The best example is between the Fibonacci functions, where n = 300, the iterative function completes this calculation near instantly: however the recursive function takes significantly longer, at about 2 minutes. Iterative is faster due to doing the whole calculation in a single loop, whereas recursive does 300 loops and takes a long time to finish.

Task 2: Understanding an efficient sorting algorithm – Merge Sort Algorithm

(1)

Test Case 1:

```
Insertion Sort took: 0.002198 seconds
Merge Sort took: 0.001946 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7, 9, 8, 3, 5, 8]

[2, 3, 3, 4, 5, 5, 5, 6, 6, 6, 7, 8, 8, 9, 9]
[1]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4]
3240
```

Test Case 2:

```
Insertion Sort took: 0.001790 seconds
Merge Sort took: 0.001636 seconds
[2, 6, 9]

[2, 6, 9]
[1]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6]
3240
```

Test Case 3:

```
Insertion Sort took: 0.002157 seconds
Merge Sort took: 0.001688 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7, 9, 8, 3, 5, 8, 5, 6, 8, 7, 9, 8, 3, 5, 8, 2, 6, 3, 6, 7, 9, 8, 9, 3, 5, 8]

[2, 2, 3, 3, 3, 3, 3, 4, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 7, 7, 7, 8, 8, 8, 8, 8, 8, 8, 9, 9, 9, 9, 9]
[1]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6]
3240
```

(2)

Test Case 1: list size between 1 and 100k for i in range(1000) looking for 50

```
Insertion Sort took: 1.045051 seconds
Merge Sort took: 0.077762 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7, 9, 8, 3, 5, 8]

[2, 3, 3, 4, 5, 5, 5, 6, 6, 6, 7, 8, 8, 9, 9]
```

Test Case 2: list size between 1 and 100k for i in range(1000) looking for 90

```
Insertion Sort took: 1.650765 seconds
Merge Sort took: 0.158879 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7, 9, 8, 3, 5, 8]

[2, 3, 3, 4, 5, 5, 5, 6, 6, 6, 7, 8, 8, 9, 9]
```

Test Case 3: list between 1 and 100k for i in range(1000) looking for 30

```
Insertion Sort took: 0.618058 seconds
Merge Sort took: 0.062360 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7, 9, 8, 3, 5, 8]

[2, 3, 3, 4, 5, 5, 5, 6, 6, 6, 7, 8, 8, 9, 9]
```

(3)

Merge Sort is by far a much more efficient algorithm than Insertion Sort. As the list size increases the gap between the two algorithms becomes vast with Insertion Sort taking a full second longer than Merge Sort when looking for 90 in a list between 1 and 100k.

Task 3: Tower of Hanoi problem – Application of recursion algorithm

(1)

Test Case 1:

```
Enter number of disks: 3
The moves are:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B

Process finished with exit code 0
```

Test Case 2:

```
Enter number of disks: 4
The moves are:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Move disk 4 from A to B
Move disk 1 from C to B
Move disk 2 from C to A
Move disk 1 from B to A
Move disk 3 from C to B
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B

Process finished with exit code 0
```

(2)

Test Case 1:

```
Enter number of disks: 3
The moves are:
Number of moves: 7

Process finished with exit code 0
```

Test Case 2:

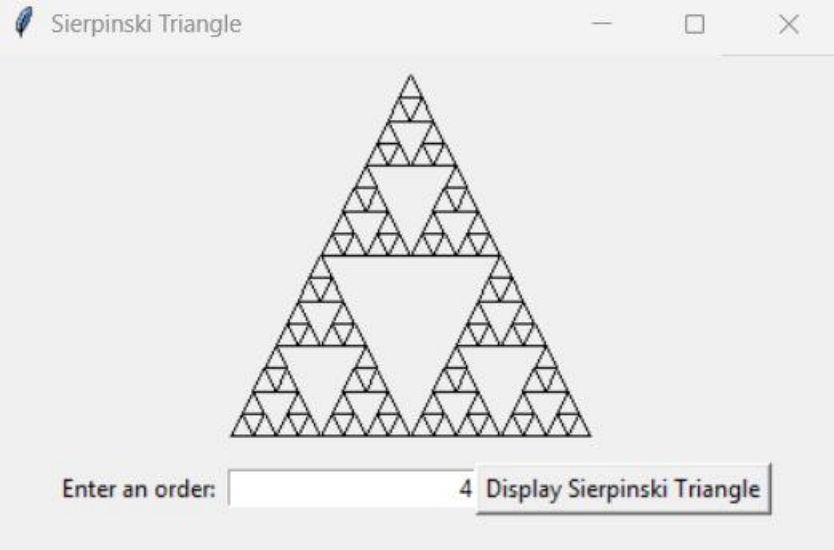
```
Enter number of disks: 7
The moves are:
Number of moves: 127

Process finished with exit code 0
```

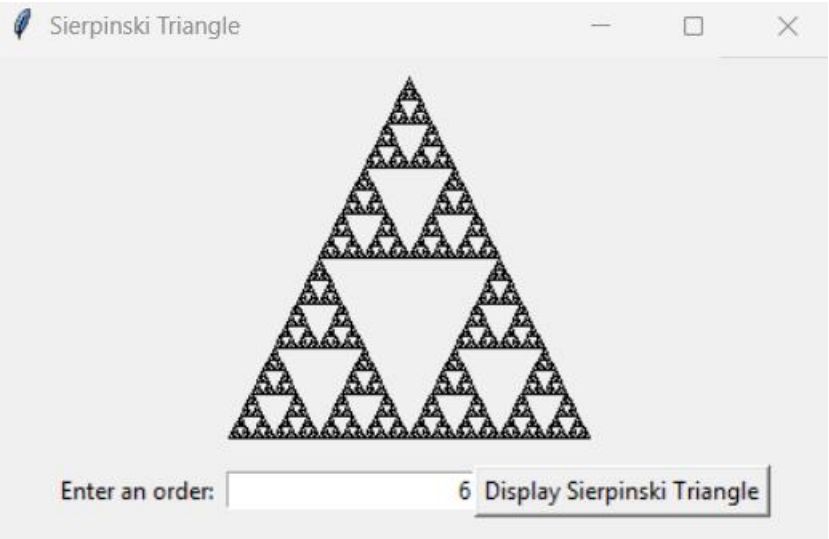
Task 4: Sierpinski Triangle – Fractals Computation using recursion algorithm

(1)

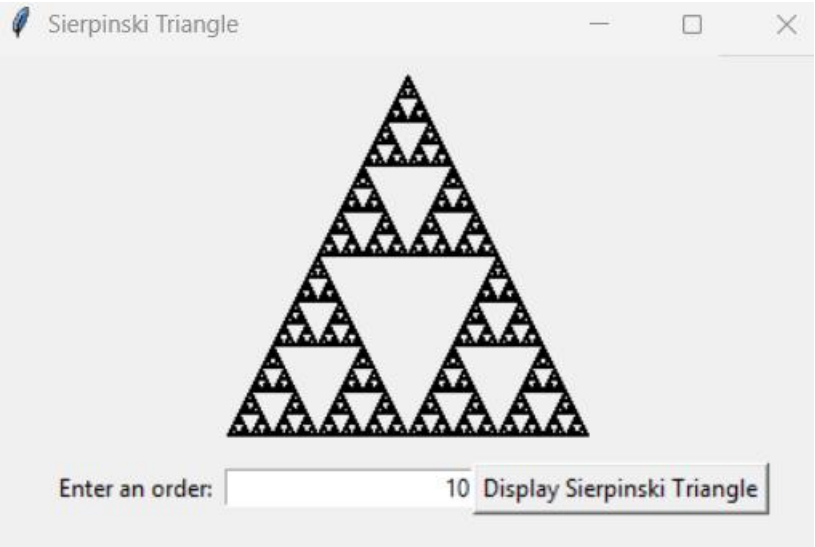
Test Case 1:



Test Case 2:

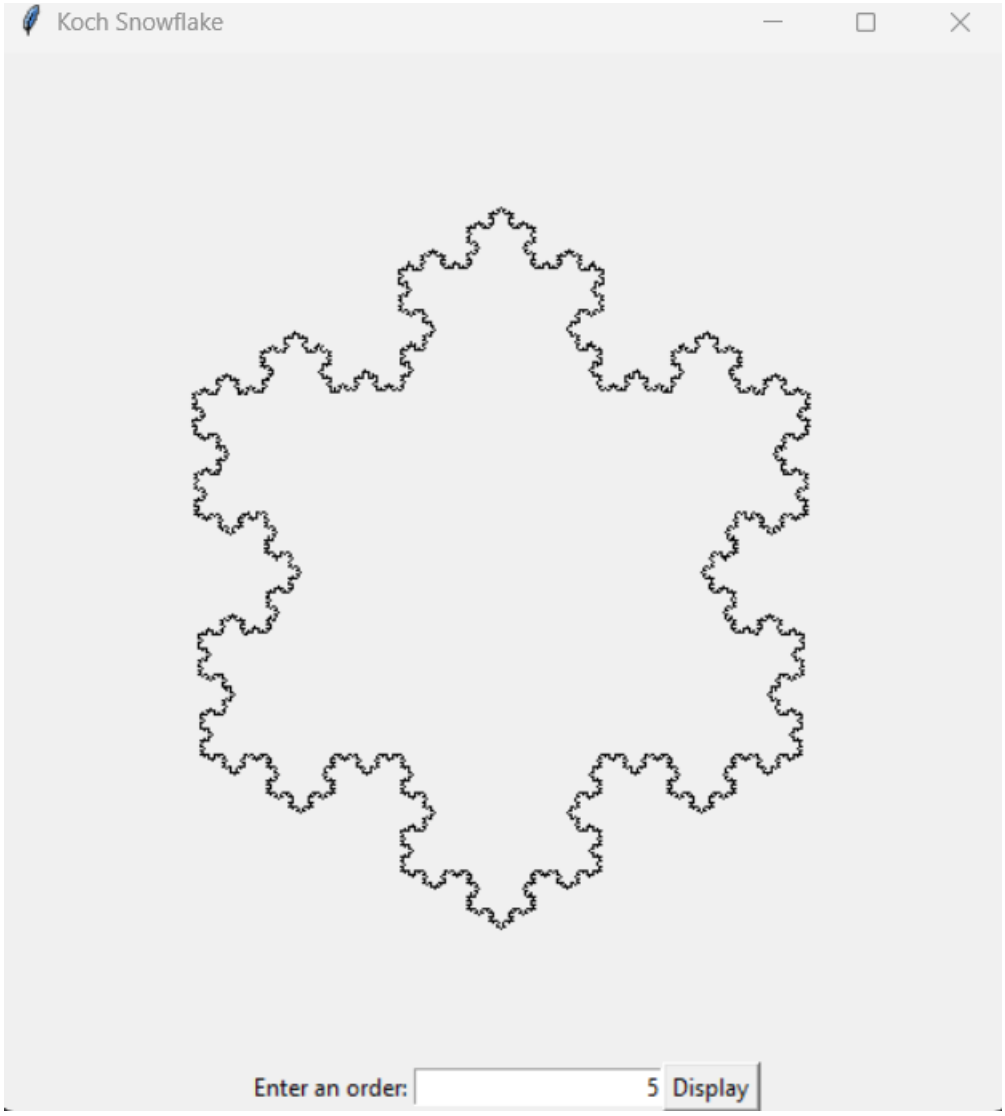


Test Case 3:

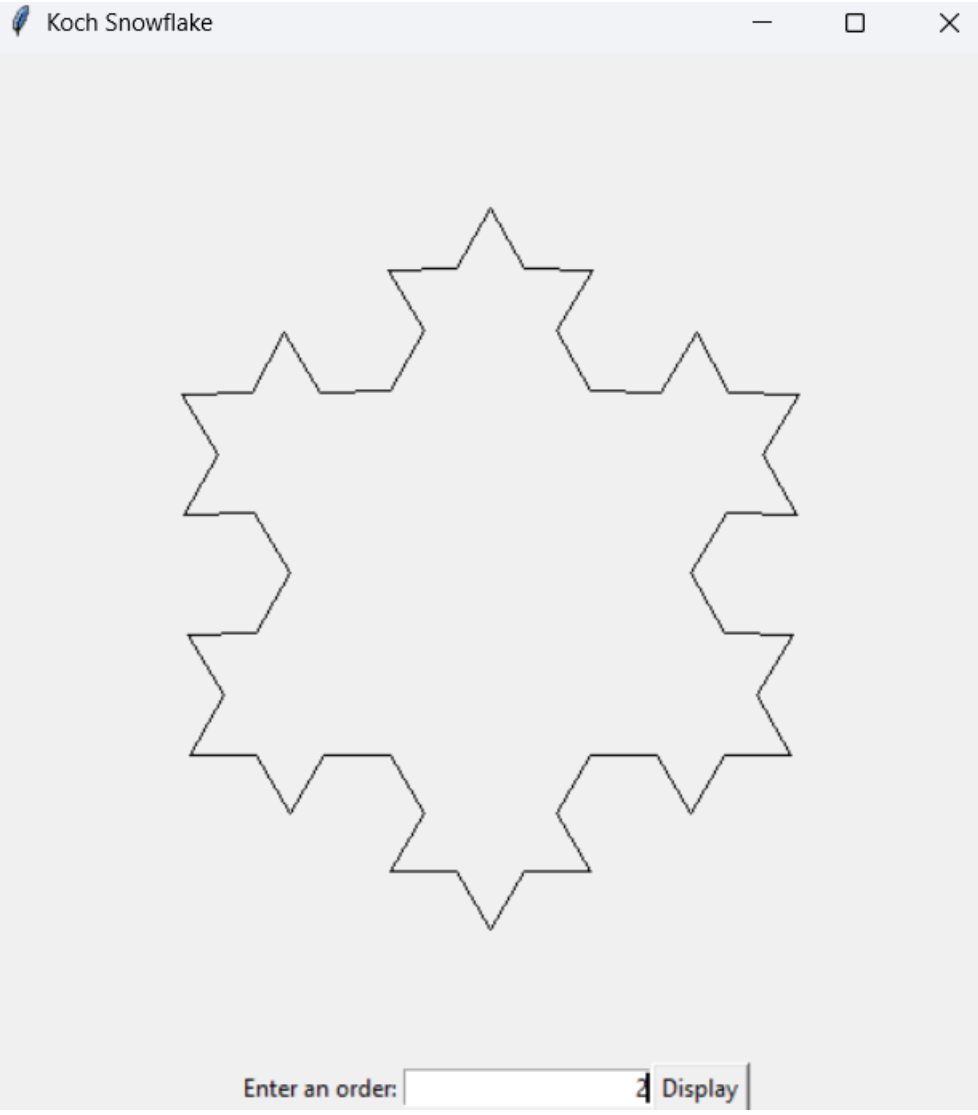

Sierpinski Triangle
Enter an order:
10
Display Sierpinski Triangle

(2)

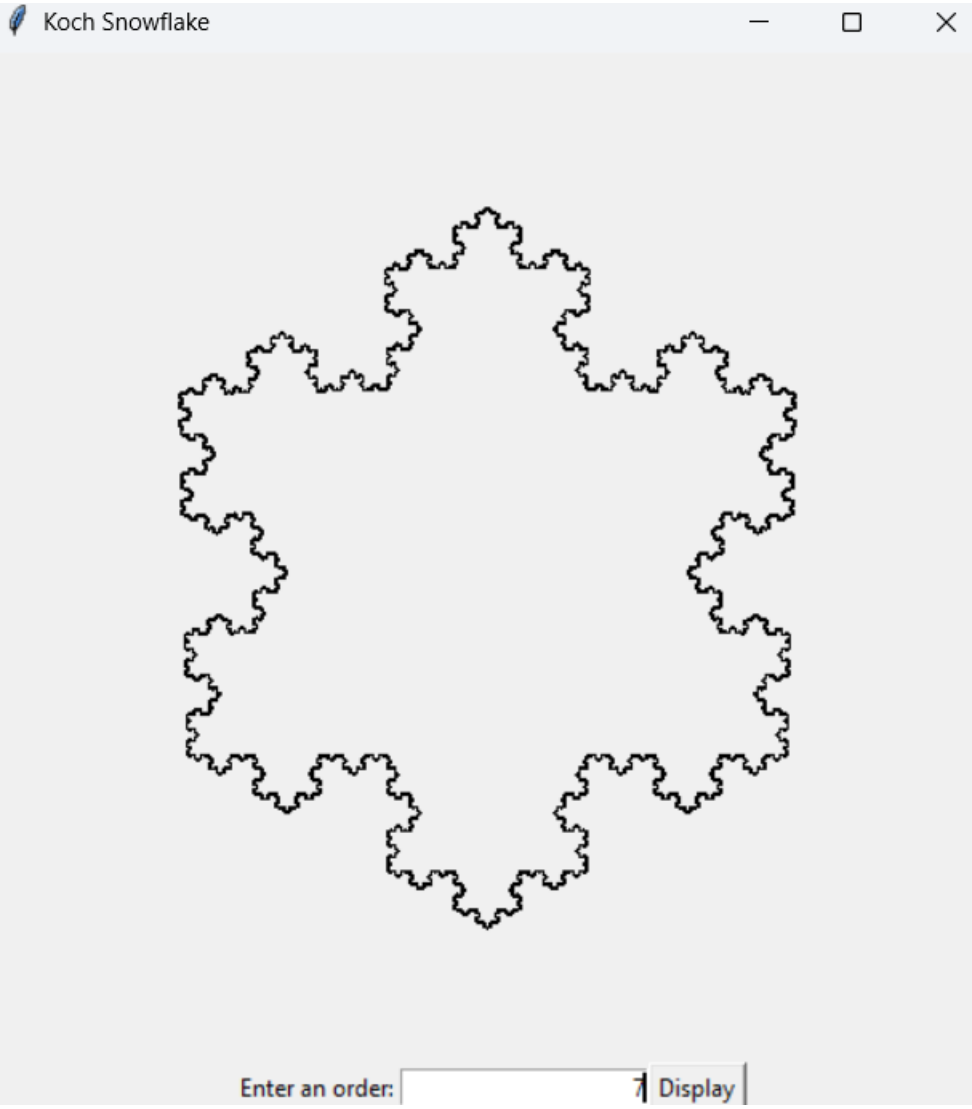
Test Case 1:



Test Case 2:



Test Case 3:


Koch Snowflake
Enter an order:
7
Display